

NITMB REU

Sophie van Genderen, Associate Computational Specialist
David Glass, Research Computing Support Lead
Haley Carter, Senior Computational Specialist
Natalie Stegman, Student Computational Consultant

Research Computing and Data Services
Northwestern IT

July 8, 2026

For a copy of these slides, visit:
github.com/nuitrcs/REU_Activities

Agenda:

- Working From the Command Line
- Git, GitHub and Version Control Systems
- HPC Systems and Quest

Working From the Command Line

Working from the command line

The command line is where you can give your computer instructions. We use words (a programming language) instead of point-and-click.

The command line language is the language used to talk to your **operating system** - move files, install programs, etc. You can also run other programming languages and software programs from the command line, if you pass the right code to start those programs.

The command line is accessible through a **terminal** application. There are many options, but the one native to Mac is Terminal and the one native to Windows is PowerShell.

Benefits and drawbacks to the command line

BENEFITS

- Explicit
- No application updates
- Necessary in some situations
- Automation
- Speed

DRAWBACKS

- Explicit
- Less user-friendly
- Limited display capabilities and other functionalities
- Most commands are permanent (no CTRL+Z or move something to trash)



Log on to Quest

1. Open a terminal application on your computer. On Macs, this is called “Terminal”. On PCs, this is called “PowerShell”.
2. Type the following command (replacing <netid> with your NetID and hit Enter):

```
ssh <netid>@login.quest.northwestern.edu
```

- if this is the first time, type yes after being prompted to fingerprint
- type your NetID password (it doesn't look like you're typing, but you are) and hit “Enter”

Command Line

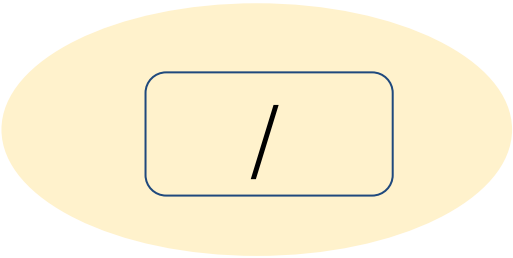
- Different terminal applications on different operating systems run different “shells” or languages to interface with the OS
- Where you can type and enter commands is called the “command prompt” and will have different symbols in different shells (\$, %, >)
- Part of the reason we’re working from Quest today is to give us a shared shell (bash) and a shared filesystem

Navigating the filesystem: pwd

Type pwd and hit “Enter”.

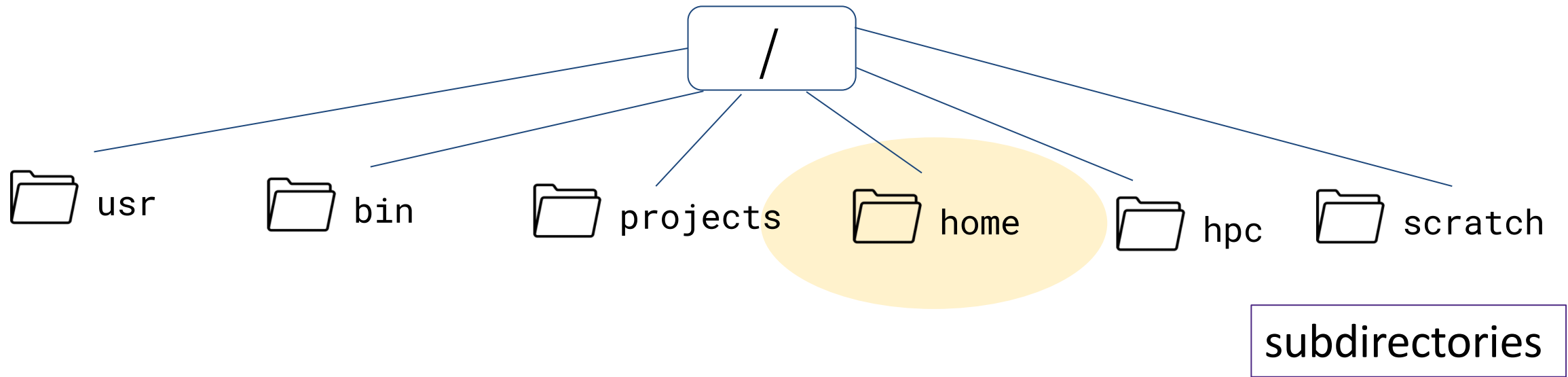
Should return: /home/<netid>

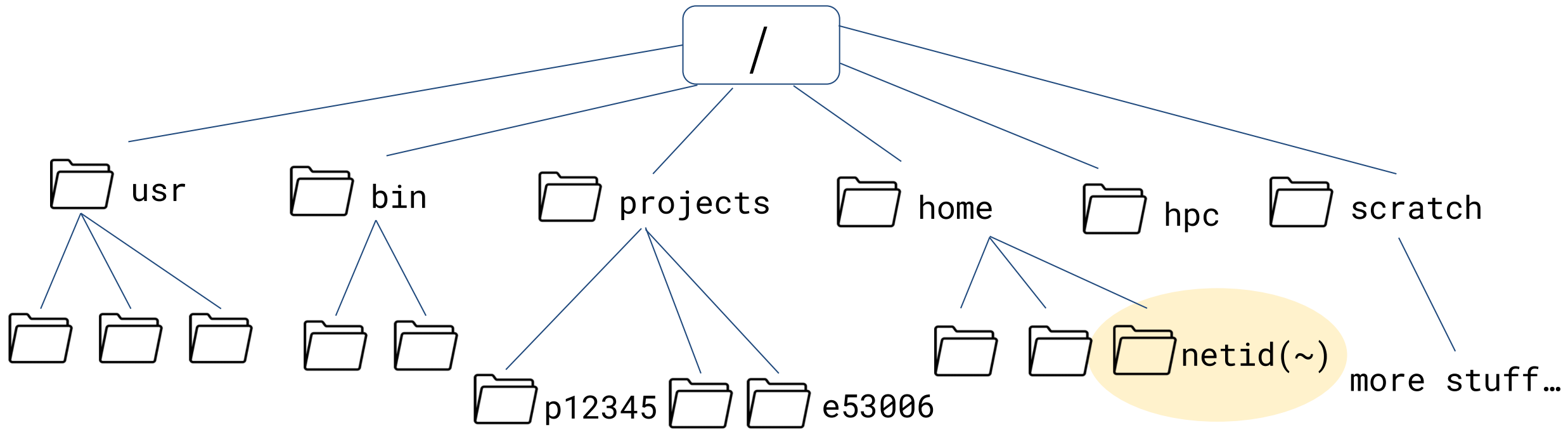
- pwd stands for print working directory (folder)
- It displays the **path** to your current location in the filesystem.

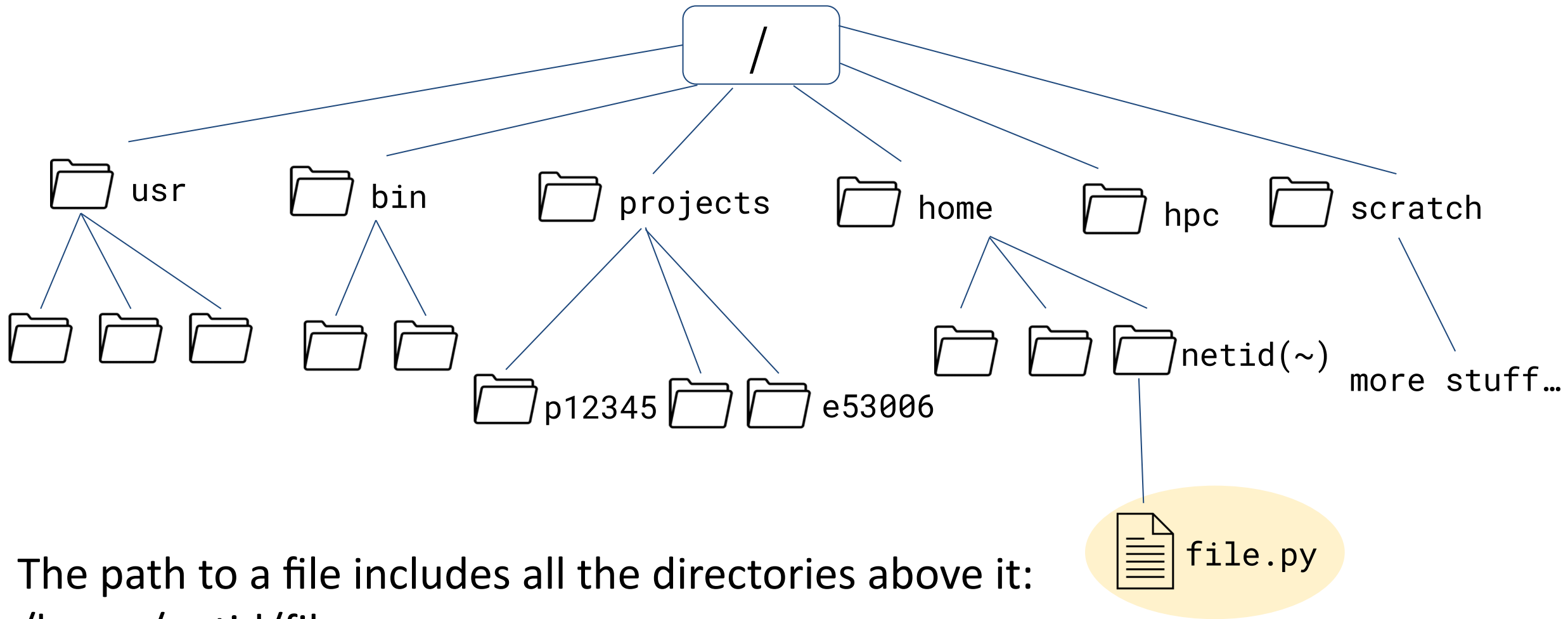


/

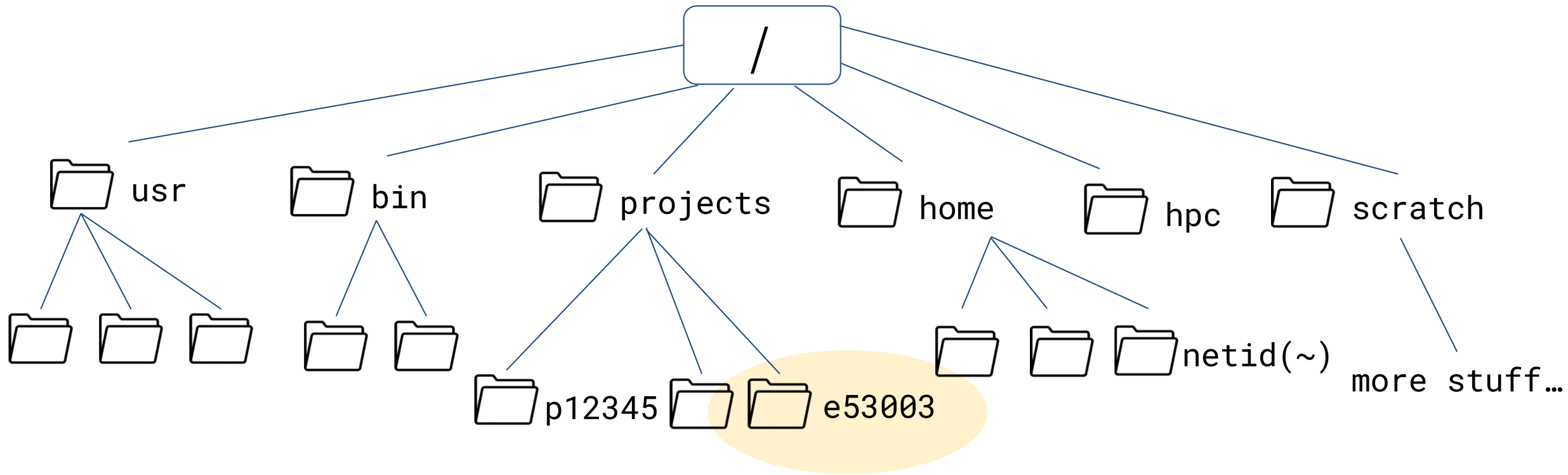
root directory







The path to a file includes all the directories above it:
`/home/netid/file.py`



We're going to move into this e53003 directory (folder). The **absolute path** is `/projects/e53003`. The **relative path** from the home directory is `../../projects/e53003`. Absolute paths start from the root of the filesystem and are unambiguous. Relative paths start in your currently location, and are therefore relative to that specific location.

Navigating the filesystem: cd

Use the following command to change locations:

```
cd /projects/e53003
```

- cd stands for change directory (folder)
- Used to move from one location to another in the filesystem
- A couple of shortcuts: cd without a location and cd ~ will both take you to your home directory, cd . . will take you one directory towards root, or “up” one directory

Navigating the filesystem: `ls`

`ls`

- `ls` stands for list
- Used to list the contents of a directory
- Without any other parameters, it will list the contents of your current working directory. But you can point it to other locations.

Navigating the filesystem: `ls`

`ls -al`

- The `-a` flag means “all” and will include hidden files.
- The `-l` flag means “long form” and will print additional information in a long form list, instead of just file and folder names.
- There are many other possible parameters, the command `man` followed by the command you’re interested in learning more about will pull up a manual for you.

A little practice...

- Make sure you're in `/projects/e53003`
- What is the content of this directory?
- Change locations into the `nitmb` directory.
- What is the content of `nitmb`?
- Run `ls -al` in the `nitmb` directory. What difference do those flags make?

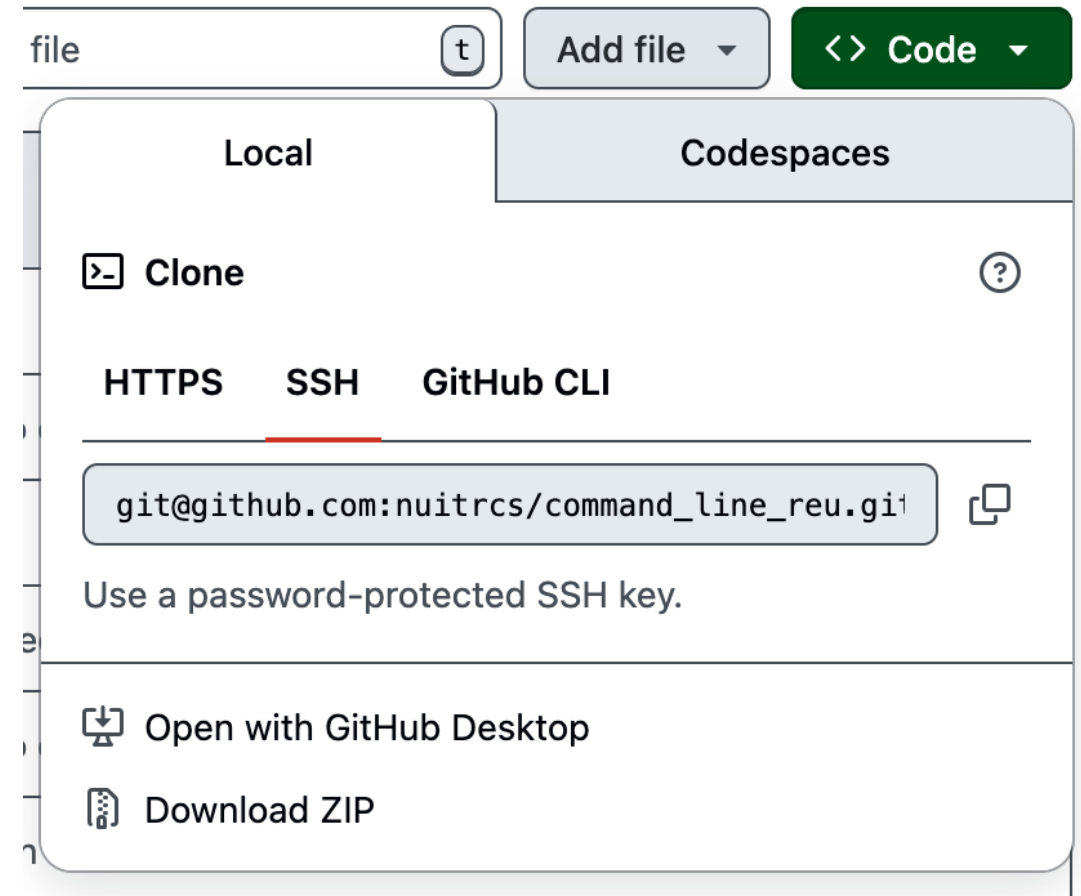
Examining files: cat, head, tail

`cat <filename>`

- Stands for concatenate
- Prints the contents of a file to the console
- `head` and `tail` will print the beginning and the end, respectively, not the whole file
- Only helpful for human-readable files (eg. `.txt`, `.md`, `.R`, `.py`) not things like `.tar`, `.gz`, `.zip`, `.jpeg`

Explore the labyrinth...

- In /projects/e53003, make a new folder with your name:
`mkdir your_name`
- Move into your new folder.
- In a browser, visit github.com/nuitrcs/command_line_reu.
 - Click the green “Code” dropdown and copy the address under HTTPS
- Back in your terminal enter the copied address into the following command:
`git clone <copied-address>`



Explore the labyrinth...

- Find the coins in the fountain in the labyrinth. What coins are in there? What is the full path to the file `nickles.md`?
- What is in the fridge in the kitchen in the hobbit house in the meadow in the wooded lane in the labyrinth?
- What are the names of our ponies? What is the full path to where the ponies live?
- How would you get from the ponies to the horses?
- Find the hidden treasure.

Commands for managing files

- `cp <source> <destination>` - copy a file from the source to the destination
- `mv <source> <destination>` - move a file from the source to the destination (also rename a file)
- `rm <file>` - PERMANENTLY delete a file (be very careful with this)
 A yellow dashed circle with a sad face inside, indicating a warning or caution.
- `touch <file>` – create a file or change timestamp of existing file

File editing from the command line: cp

```
cp </path/to/file/filename> <desired/location/>
```

- Copies a file from an existing location to a new location
- You can rename files if you provide a new name with your new location.
- You can copy an entire folder if you add -r or -R

Try it! `cp /projects/e53003/nitmb/introduction.txt`
to your new folder.

File editing from the command line: mv

```
mv <previousfilename> <newfilename>
```

- Stands for “move”.
- Can be used to move a file or rename a file.
- It is always safer to cp and then rm the old file than to mv it directly, but this is more of a concern for larger files than small pieces of text.

Try it! Rename your new copy of introduction.txt to <yourname>.txt.

Command line text editors

- Run within a terminal or command-line interface (CLI)
- Create and edit plain text files without a graphical interface or needing to repetitively upload/download files
 - Prototype and debug code
 - View program output
 - Read documentation
- Several different command line text editors available:
 - Nano
 - Vi/Vim
 - Emacs



File editing from the command line: nano

`nano <filename>`

- Opens the file in a command-line editor
- Allows you to write or edit files from within the terminal
- Ctrl+O, Ctrl+X to save and exit

Try it! From within your new folder, create a new file with nano called `notes.txt`.

File editing from the command line: nano

```
nano notes.txt
```

Write some notes about the commands we've used today. Notice that you have to use the arrow keys instead of the mouse to navigate.

Use *Ctrl+O*, *Ctrl+X* to save and exit.

A little practice...

- Open your new copy of `introduction.txt` with nano.
- Edit the file with your name and favorite color.
- Save and exit from nano.
- Copy the newly edited file into `/projects/e53003/nitmb/`

Tips for command line text editors

- There is some learning curve, but once you internalize the key bindings, they become second-nature
- Use your arrow keys to navigate (not your mouse)
- Save often!
- Consult the user guide/manual
 - <https://www.nano-editor.org/dist/latest/cheatsheet.html>
- If you are trying something new, back up important files before drastically editing them

Utility commands

- `Ctrl + C` – stop current command execution
- `find <start_dir> -maxdepth <num> -name <filename>` – recursively search the filesystem at specified level of recursion for a file
- `grep <pattern> <filename>` - searches the specified file for a pattern
- `man <command>` – display the manual for the specified command
- `top` – display all running processes
- `history` – print your command history
- `Ctrl + R <phrase>` - reverse-i-search your command history for a phrase

Utility commands

- `find <start_dir> -maxdepth <num> -name <filename>`
recursively search the filesystem at specified level of recursion for a file
- There are three “arguments” or “flags” or “parameters” outlined in the command above. The first `<start_dir>` is **positional**, meaning it is identified by its position in relationship to the command. It must come first after `find`. The other two are **named**, meaning they are defined by names: `-maxdepth` and `-name`. Generally, named arguments are order irrelevant. `-maxdepth` is an **optional** argument. `-name` is a **required** argument.

Utility commands

- `find <start_dir> -maxdepth <num> -name <filename>` recursively search the filesystem at specified level of recursion for a file

Try it out!

- We're looking for a file called `exercise_1.md` which is somewhere within the labyrinth directory.
- Once you find it, follow the instructions in the file.

Extra Practice

There are additional exercise files within the `command_line_reu` folder that you cloned down:

- `exercise_0.md` - filesystem navigation
- `exercise_2.md` - nano, including key shortcuts

If you find yourself with extra time, during this workshop or afterwards, feel free to look for and then work through any of these files!

Who will have access to my files?



Files created in
/projects



Read/write permissions to all
project members by default



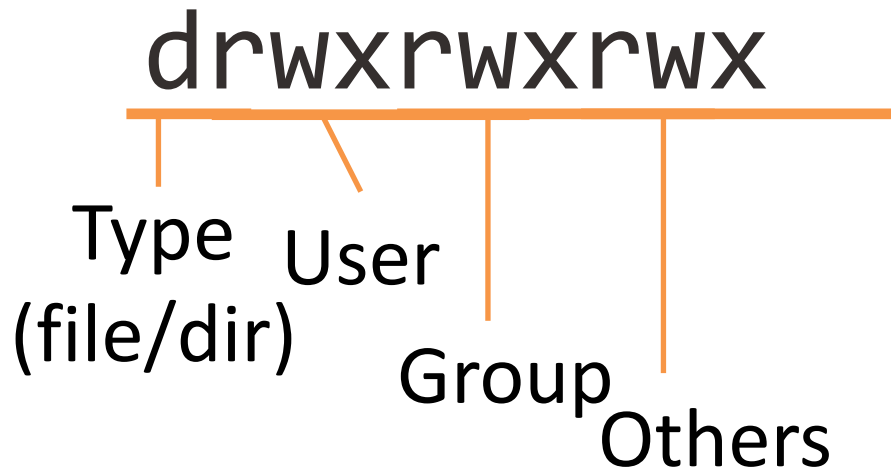
Files created in
/home



Read/Write permissions only
you by default

Permission strings

```
[abc1234@quser21 p12345]$ ls -l
total 2
drwxrwxr-x 1 abc1234 p12345 4096 Apr 15 11:00 codes
drwxrwxr-x 1 abc1234 p12345 4096 Apr 15 10:00 data
-rw-rw-r-- 1 abc1234 p12345  259 Apr 15 09:00 test.txt
```



- Who can edit files or make a new file within codes?
- Who can execute files within codes?
- Who can read test.txt?
- Who can edit test.txt?

Best practices and tips for the command line

- Every character is important
 - Watch out for case-sensitivity and typos
- Permanency of commands (deleted files are often gone forever)
- Use Tab to auto-complete paths
- Use the  key to scroll through command history
- Use CTRL+R to search through command history
- Use CTRL+C to stop/cancel the execution of a command
- Avoid spaces and special characters in file and folder names when possible
 - Or type \ before each space to specify the path
- Consult the man pages of commands you're using
- Be cautious, but don't be afraid to try things out!

Git, GitHub and Version Control Systems

Outline

- By the end of this training, you will understand...
 - The motivation for Git and Version Control Systems in General
 - How Git Works
 - How to make a ***commit***
 - How to perform a ***merge***
 - How to create a ***Pull Request***

Version Control Systems (VCS)

- Essential for managing software development projects
- Helps teams track code changes
- Allows multiple people working on the same project at once
- Protects the source code from unintended human errors (if done correctly)

Git vs GitHub

- Software vs Service
- **Git**: locally installed version control system
- **GitHub**: Cloud-based platform that hosts Git repositories and provides additional collaboration features, as well as tracking issues and changes
- Git is the underlying technology that GitHub uses to store and manage code

Git Basics

Terminology

- **Commit:** also called a "revision", is a change made to a local file or multiple files. You make a commit to save your work
 - It is encouraged to write a commit message – a brief explanation of the changes made
- **Push:** you send your changes (commits) to a repository on GitHub.com
 - Ex: you change something to a file locally and can then push the changes so that your collaborators can see them too.

Terminology

- **Pull Request:** you grab changes someone else made and merge them with your copy (or with the origin file)
 - Ex: you work on a file with someone and they edited the file, you will **pull** those changes to your local copy
- **Pull vs Fetch**
 - Pull immediately merges the changes in with your copy
 - Fetch allows you to review the changes before you commit them

Terminology

- **Branch:** a separate line of development within a repository that can be merged in when the changes are ready
- **Fork:** a full copy of a repository under the copier's account – you can merge the fork back into the original repository if you have permissions

Terminology

- **Merge:** Takes the changes from one branch and applies them to another (within the same repository)
 - This often happens as a "pull request" (the request to merge) or through the command line

Ok, but how do we use it?

STEP 1 OF 2

The upstream repository

The shared, canonical repo lives on GitHub — the single source of truth that everyone works from and contributes back into.

source ·

github.com/nuitrcs/github-reu

GITHUB · UPSTREAM

● **nuitrcs / github-reu**

UPSTREAM

main



awaiting clones

YOUR COMPUTER · LOCAL CLONES

nothing cloned yet

STEP 2 OF 2

Clone it to your computer

Cloning downloads the repository from GitHub onto a machine, giving you a local working copy. Clone onto as many machines as you like.

```
$ git clone https://github.com/nuitrcs/github-reu.git
```

GITHUB · UPSTREAM

● **nuitrcs / github-reu**

UPSTREAM

git clone ↓

YOUR COMPUTER · LOCAL CLONES

■ **Laptop** · github-reu

LOCAL CLONE

■ **Quest** · HPC cluster

LOCAL CLONE

STEP 2 OF 2

Clone it to your computer

Cloning downloads the repository from GitHub onto a machine, giving you a local working copy. Clone onto as many machines as you like.

```
$ git clone https://github.com/nuitrcs/github-reu.git
```

GITHUB · UPSTREAM

● **nuitrcs / github-reu**

UPSTREAM

git clone ↓

YOUR COMPUTER · LOCAL CLONES

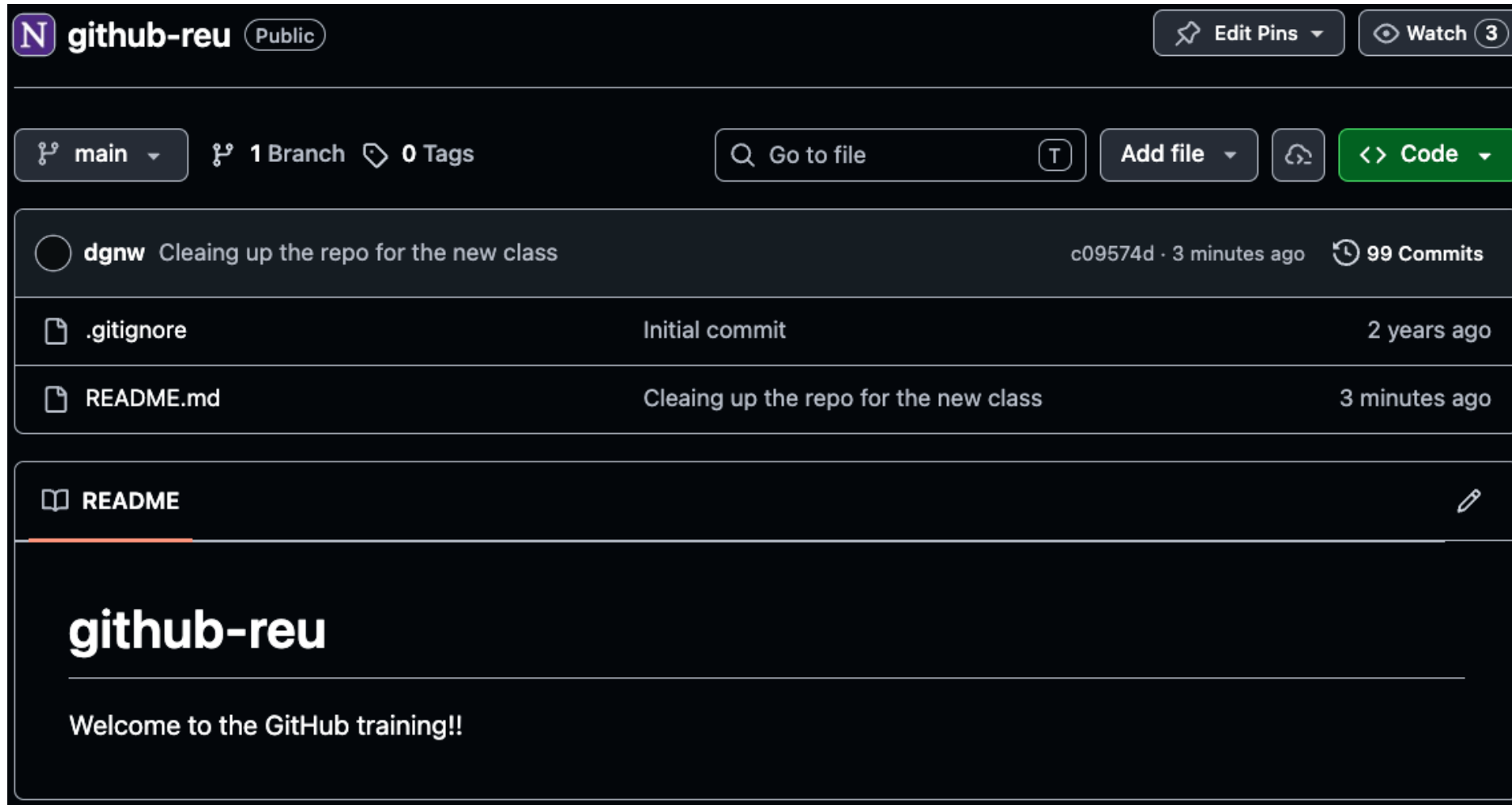
■ **Laptop** · github-reu

LOCAL CLONE

■ **Quest** · HPC cluster

LOCAL CLONE

Our Upstream Repository



- <https://github.com/nuitrcs/github-reu>

Setup to Clone (ssh keys)

- What is an ssh key?
 - An authentication tool that allows GitHub to confirm your identity.
- How do I create one?

```
# mac or linux
```

```
ssh-keygen -a 100 -t ed25519 -f ~/.ssh/id_ed25519 -C "comment here"
```

```
# windows
```

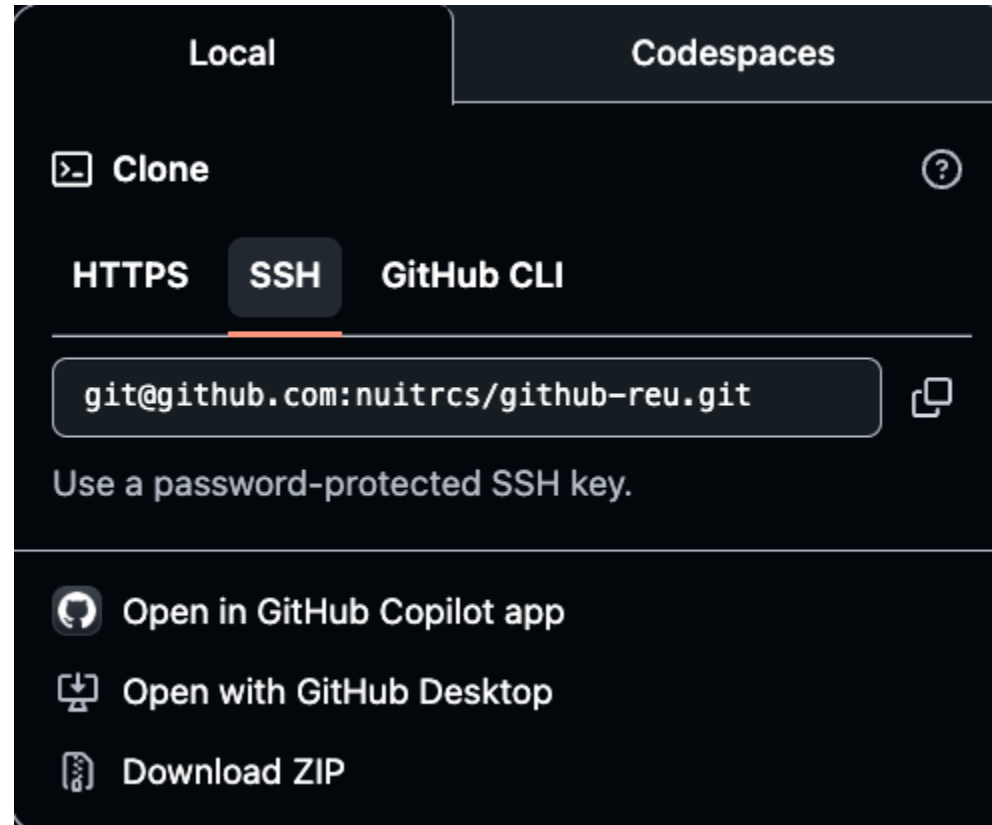
```
ssh-keygen -t ed25519
```

<https://www.purdue.edu/science/scienceit/ssh-keys-windows.html>

Adding ssh to GitHub

1. On your laptop, navigate to the .ssh directory and open the file that ends in ".pub".
2. Copy this output from end-to-end.
3. Log in to GitHub.com and click on your profile in the upper-right.
4. Select "Settings", then on the left menu select "SSH and GPG Keys".
5. Click the "New SSH Key" button and paste in the contents of the ".pub" file.
6. Now you can use ssh!

Clone a Repository



- <https://github.com/nuitrcs/github-reu>

Clones!

- Different protocols:
 - ssh
 - Requires setting up ssh keys
 - One-time setup
 - http
 - Requires a personal access token (PAT)
 - One-time setup
- Different tools:
 - Command line git
 - GitHub desktop application



Return of the Clones!

```
$ git clone git@github.com:nuitrcs/github-reu.git
```

```
Cloning into 'github-reu'...
```

```
Warning: Permanently added the ECDSA host key for IP address '140.82.114.3' to the  
list of known hosts.
```

```
remote: Enumerating objects: 165, done.
```

```
remote: Counting objects: 100% (33/33), done.
```

```
remote: Compressing objects: 100% (27/27), done.
```

```
remote: Total 165 (delta 8), reused 27 (delta 5), pack-reused 132 (from 1)
```

```
Receiving objects: 100% (165/165), 69.08 KiB | 2.23 MiB/s, done.
```

```
Resolving deltas: 100% (37/37), done.
```

STEP 2 OF 2

Clone it to Quest

Cloning downloads the repository from GitHub onto a machine, giving you a local working copy. Clone onto as many machines as you like.

```
$ git clone https://github.com/nuitrcs/github-reu.git
```

GITHUB · UPSTREAM

● **nuitrcs / github-reu**

UPSTREAM

git clone ↓

YOUR COMPUTER · LOCAL CLONES

■ **Laptop** · github-reu

LOCAL CLONE

■ **Quest** · HPC cluster

LOCAL CLONE

Ways of Working in Git

Branch

- A separate duplication of the code that you can make changes to.
- Why branch?
 - Repositories often have many collaborators.
 - If everyone pushes to "main", conflicts occur.
 - Work is lost; people get angry, bad things happen.

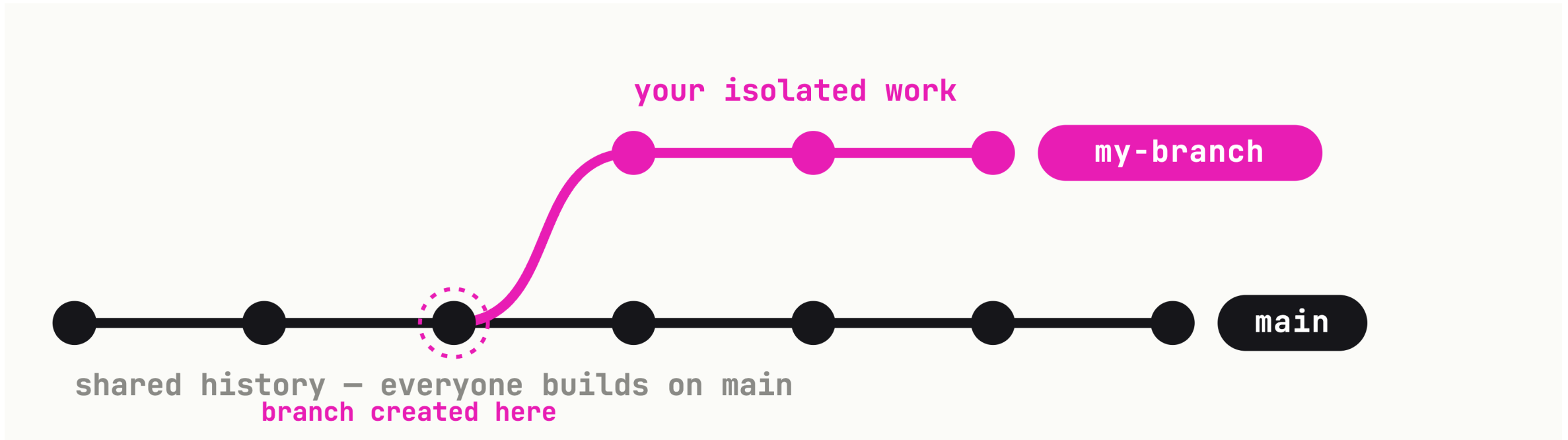
```
$ git checkout -b "example_branch"  
Switched to a new branch 'example_branch'
```

What is a branch?

```
$ git checkout -b "my-branch"
```

Switched to a new branch 'my-branch'

A branch is a parallel line of work inside one repository — a copy of the code you can change freely. You commit to **your branch** while **main** stays stable and conflict-free. Then you merge back into main when ready.



- Commits on **my-branch** don't touch **main** until you merge them back — so you can experiment without breaking everyone else's work.

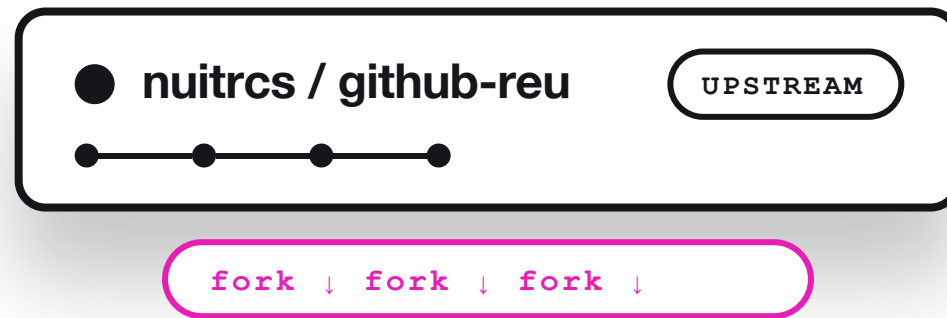
Forking

- Forking creates a separate copy of the repository.
- Branching creates a parallel work stream in the existing repository.
- Which one should I use?
 - Forking is best for building on top of existing code.
 - Branching is best for editing or contributing to existing repositories.

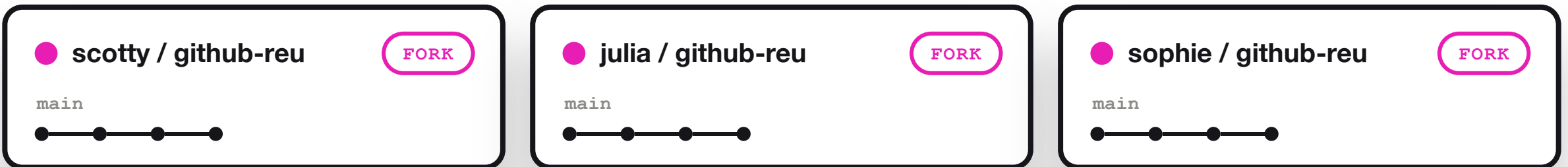
Forking makes a separate copy

A **fork** is a whole separate repository under a new owner. A **branch** is a parallel stream inside one repo. Fork to build on top; branch to contribute.

GITHUB · UPSTREAM



GITHUB · FORKS — one separate copy per person



Commit and Push

```
(base) [euz9115@quser34 github-reu]$ git checkout -b "edits_sophie"
Switched to a new branch 'edits_sophie'
(base) [euz9115@quser34 github-reu]$ ls
README.md
(base) [euz9115@quser34 github-reu]$ vi README.md
(base) [euz9115@quser34 github-reu]$ git add .
(base) [euz9115@quser34 github-reu]$ git commit -m "Sophie added her edits"
[edits_sophie 6d9199f] Sophie added her edits
1 file changed, 3 insertions(+)
(base) [euz9115@quser34 github-reu]$ git push
fatal: The current branch edits_sophie has no upstream branch.
To push the current branch and set the remote as upstream, use

    git push --set-upstream origin edits_sophie

To have this happen automatically for branches without a tracking
upstream, see 'push.autoSetupRemote' in 'git help config'.
(base) [euz9115@quser34 github-reu]$
```

Commit and Push – Error Message

```
(base) [euz9115@quser34 github-reu]$ git push
fatal: The current branch edits_sophie has no upstream branch.
To push the current branch and set the remote as upstream, use

    git push --set-upstream origin edits_sophie

To have this happen automatically for branches without a tracking
upstream, see 'push.autoSetupRemote' in 'git help config'.

(base) [euz9115@quser34 github-reu]$ git push --set-upstream origin edits_sophie
Warning: Permanently added 'github.com,140.82.112.3' (ECDSA) to the list of known hosts.
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 52 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 345 bytes | 345.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote:
remote: Create a pull request for 'edits_sophie' on GitHub by visiting:
remote:   https://github.com/nuitrcs/github-reu/pull/new/edits_sophie
remote:
To github.com:nuitrcs/github-reu.git
 * [new branch]      edits_sophie -> edits_sophie
branch 'edits_sophie' set up to track 'origin/edits_sophie'.
(base) [euz9115@quser34 github-reu]$ git push
Warning: Permanently added 'github.com,140.82.112.3' (ECDSA) to the list of known hosts.
Everything up-to-date
(base) [euz9115@quser34 github-reu]$
```

REMOTES

Every clone has a remote: **origin**

A **remote** is a named link to a repo on GitHub. When you clone, git automatically adds one called **origin** — pointing back at the fork you cloned from.

```
$git remote -v
origin git@github.com:scotty/github-reu.git
```

GITHUB · UPSTREAM

● nuitrcs / github-reu

UPSTREAM

GITHUB · YOUR FORK

● scotty / github-reu

YOUR FORK



↑ **origin**

every clone's **origin** points back to your fork

YOUR COMPUTER · LOCAL CLONES

■ Laptop

LOCAL CLONE

remote: **origin** → scotty

■ Quest · HPC cluster

LOCAL CLONE

remote: **origin** → scotty

Pull Request

<> Code

Issues

Pull requests 1

Actions

Projects

Wiki

Security

Insights

Open a pull request

Create a new pull request by comparing changes across two branches. If you need to, you can also [compare across forks](#). [Learn more about diff comparisons here](#).

base: main

compare: edits_sophie

✓ Able to merge. These branches can be automatically merged.

Add a title

Sophie added her edits

Add a description

Write

Preview

H B I @

Add your description here...

Markdown is supported

Paste, drop, or click to add files

Reviewers

Suggestions

scottcoughlin2014

Assignees

No one—[assign yourself](#)

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Use [Closing keywords](#) in the descr
automatically close issues

Create pull request

Merge

Sophie added her edits #43

Edit



svangenderen wants to merge 1 commit into `main` from `edits_sophie`



Conversation 0



Commits 1



Checks 0



Files changed 1



svangenderen commented [now](#)

Member



No description provided.



[Sophie added her edits](#)

6d9199f



No conflicts with base branch

Merging can be performed automatically.

Merge pull request



You can also merge this with the command line. [View command line instructions.](#)



Add a comment

Reviewers

Suggestions



[scottcoughlin2014](#)

Still in progress? [Convert to draft](#)

Assignees

No one—[assign yourself](#)

Labels

None yet

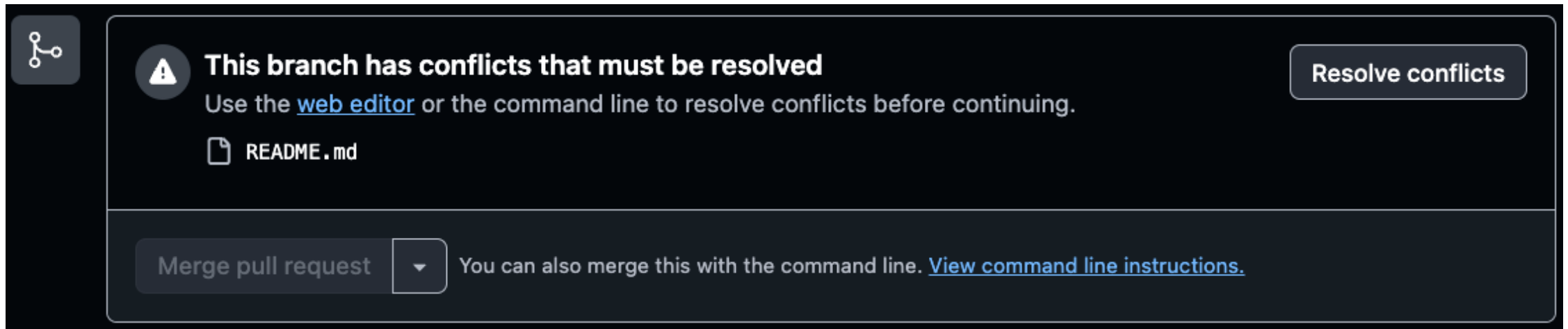
Projects

None yet

Git Conflicts

Merge Conflicts

- What happens if two or more people make changes to the same thing in the repository?
 - Merge conflicts!!



One repo, one contested line

Alex and Sam both cloned the same repo and ran `git pull`. Everyone is in sync. The repo has one file — the team's contested recipe.

■ chicago-dog.md

The Perfect Chicago-Style Hot Dog

Toppings, in order

- yellow mustard
- chopped white onions
- bright green sweet relish
- a dill pickle spear, tomato wedges, sport peppers
- your one bonus topping goes here 🍌



The trap

Both students are about to edit **the exact same line** — that bonus topping. That's the one thing Git can't auto-merge.

Two students, two edits, same line

Alex merges first. Sam — who **never pulled** afterward — edits the same line on a branch built from the old `main`.

A Alex → merged to main ✓

```
$ git switch -c alex-topping  
$ git commit -am "add celery salt"
```

+ a dash of celery salt

S Sam → stale branch ⚠

```
$ git switch -c sam-topping  
$ git commit -am "add ketchup"
```

+ a squirt of ketchup 🍅

Two different edits to the same line. Git has no way to know which one wins.

💥 git pull → CONFLICT

Sam finally pulls. Git merges everything cleanly... until that one line, where it gives up and writes **conflict markers** right into the file.

```
$ git pull origin main
Auto-merging chicago-dog.md
CONFLICT (content): Merge conflict
in chicago-dog.md
Automatic merge failed; fix
conflicts and then commit.
```

chicago-dog.md

<<<<<< HEAD

- a dash of celery salt

=====

- a squirt of ketchup 🍅

>>>>>> sam-topping

Don't panic — this is Git being helpful. It's showing both versions and asking *you* to choose.

Read the markers, pick a side

<<<<<< HEAD

Your side — what's on main (celery salt).

=====

The divider between the two versions.

>>>>>> sam-topping

The incoming side (Sam's ketchup).

Delete the markers and keep only the line you want. The Chicago rule is clear: **never ketchup.**

resolved ✓

- a dash of celery salt 🥨

VS Code even gives you *Accept Current / Incoming / Both* buttons that delete the markers for you.

Finish the merge

```
$ git add chicago-dog.md
$ git commit -m "resolve: celery salt"
$ git push
✓ Merge complete. Everyone back in sync.
```



You created a conflict on purpose and resolved it. Conflicts aren't failures — just Git asking a question only a human can answer.

Three golden rules to avoid the next one



Pull before you push

Sam's whole mess came from skipping this.



Small, frequent commits

Less overlap, easier to untangle.



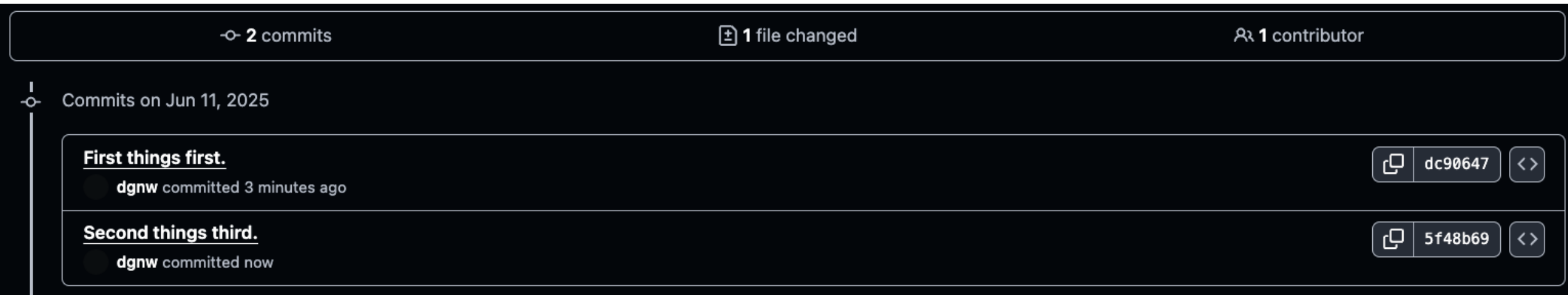
Talk to your team

"I'm editing the recipe" saves grief.

Draft PRs

Draft PRs

- Draft PRs -> a way to create a work in-progress PR.
- Allows user(s) to continue making changes without mistakenly merging them.



The screenshot shows a GitHub pull request interface. At the top, a dark header bar contains the text "2 commits", "1 file changed", and "1 contributor". Below this, a section titled "Commits on Jun 11, 2025" lists two draft commits. The first commit, titled "First things first.", was committed by "dgnw" 3 minutes ago and has a commit hash of "dc90647". The second commit, titled "Second things third.", was committed by "dgnw" "now" and has a commit hash of "5f48b69". Each commit entry includes a copy icon, the commit hash, and a compare/patch icon.

2 commits 1 file changed 1 contributor

Commits on Jun 11, 2025

First things first.  dc90647 

 **dgnw** committed 3 minutes ago

Second things third.  5f48b69 

 **dgnw** committed now

Draft PRs

Add a title

draft PR example

Add a description

WritePreview

H B I | | @

Add your description here...

Markdown is supported

Paste, drop, or click to add files

Create pull request

Remember, contributions to this repository should follow our [GitHub Community Guidelines](#)

1 commit

1 file

✓ Create pull request

Open a pull request that is ready for review

Create draft pull request

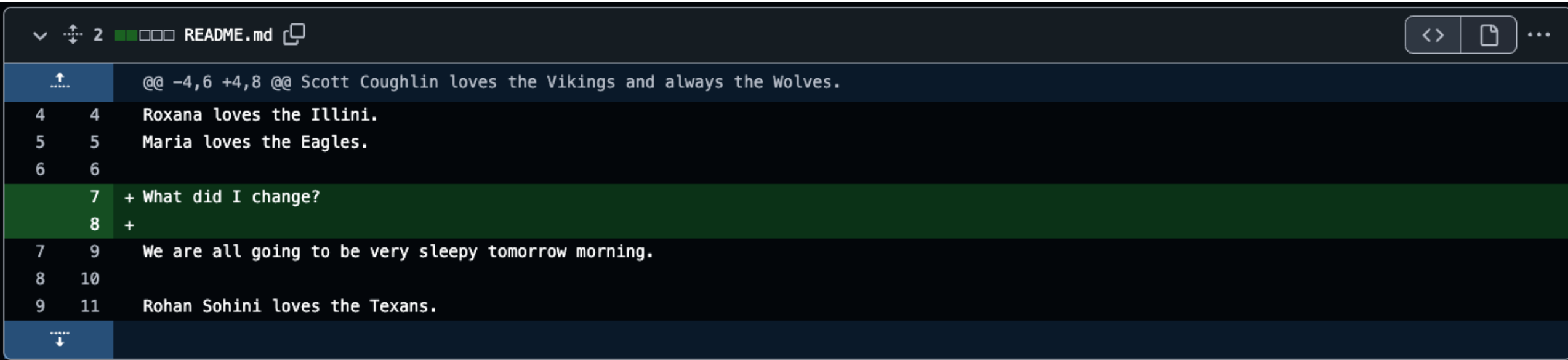
Cannot be merged until marked ready for review

Helpful Tips

Diff

- What do I do if I forget all that I've changed?
 - Use diff!
- It's a helpful way to compare the current branch with any other branch.

Diff – GitHub Online



The screenshot shows a GitHub diff interface for a file named `README.md`. The interface has a dark theme. At the top, there's a header bar with a dropdown arrow, a diff icon, the number '2', a progress bar, and the file name 'README.md' with a copy icon. On the right side of the header, there are icons for code, a document, and a menu. The main area displays the diff content. It starts with a blue bar containing an upward arrow icon. The diff shows several lines of text. Line 4 is 'Roxana loves the Illini.' and line 5 is 'Maria loves the Eagles.'. Lines 7 and 8 are highlighted in green, indicating additions: '+ What did I change?' and '+'. Line 9 is 'We are all going to be very sleepy tomorrow morning.' and line 11 is 'Rohan Sohini loves the Texans.'. The diff uses standard symbols: '-' for deletions, '+' for additions, and space for context. The bottom of the diff view has a blue bar with a downward arrow icon.

Line	Diff	Content
@@ -4,6 +4,8 @@		Scott Coughlin loves the Vikings and always the Wolves.
4		Roxana loves the Illini.
5		Maria loves the Eagles.
6		
7	+	What did I change?
8	+	
7		We are all going to be very sleepy tomorrow morning.
8		
9		Rohan Sohini loves the Texans.

Diff – Git Command Line

```
$ git diff diff_branch main
diff --git a/README.md b/README.md
index b2bc9de..4d9c9b4 100644
--- a/README.md
+++ b/README.md
@@ -4,8 +4,6 @@ Scott Coughlin loves the Vikings and always the Wolves.
Roxana loves the Illini.
Maria loves the Eagles.

-What did I change?
-
We are all going to be very sleepy tomorrow morning.

Rohan Sohini loves the Texans.
```

Demo

Your Turn!

- Edit the README in the repository
- Commit your changes
- Ask us if you have any questions!

Introduction to HPC and Quest

Northwestern

Overview

1. What is High-Performance Computing?
2. Make-up of Quest
3. SLURM Job Scheduler
4. Types of Jobs
 - Submit a batch job to Quest
 - Submit a Quest OnDemand job
5. Checking on Jobs with Slurm
6. File transfers and Storage Practices
7. Software on Quest
8. TLDR

High-Performance Computing

HPC uses supercomputers and computer clusters to solve advanced computation problems. – Wikipedia

HPC encompasses solutions that are able to process data and execute calculations at a rate that far exceeds other computers. This aggregate computing power enables [scientists] to solve large problems that would otherwise be unapproachable. – HPE

HPC is technology that uses clusters of powerful processors, working in parallel, to process massive multi-dimensional datasets (big data) and solve complex problems at extremely high speeds. - IBM

HPC at Your Institution

- What is available at your university?
- Are there multiple clusters?
 - What are their different use cases?
- Do you have to pay to use HPC resources at your university?
 - Does that include GPUs?
- What scheduler does your university use?
 - Are there different partitions? How are they defined?
- How much storage is available to each user?

Take 10 minutes to research these questions for your home university.

You may be asked to share with the class.

Northwestern's HPC Cluster = Quest

- What is available at your university? Quest.
- Are there multiple clusters? Not yet.
 - What are their different use cases? Eventually there will be a GPU cluster.
- Do you have to pay to use HPC resources at your university? No, though researchers can pay for additional storage and exclusive nodes.
- What scheduler does your university use? Slurm.
 - Are there different partitions? How are they defined?
 - General access partitions are divided by job run time, and resource type:
 - short - 4 hour limit, CPU only
 - normal - 48 hour limit, CPU only
 - long - 7 day limit, CPU only
 - gengpu – 48 hour limit, GPU only
 - genhmem – 48 hour limit, >473GB memory per node
- How much storage is available to each user?
 - 80GB in home directory
 - 1-2 TB of project storage
 - More available for purchase

Accessing remote systems

- Terminal / Command line
 - `ssh <username>@<domain.name>`
- OnDemand browser-based access especially for GUI-based software (graphical user interface)
 - `https://ondemand.quest.northwestern.edu`

When Do You Use an HPC?

- Training Large AI Models
 - Models have billions of parameters
 - Training requires huge processing datasets
 - Needs GPUs/accelerators working in parallel
- Genomics and DNA Sequencing
 - Human genome = ~3 billion base pairs
 - Modern sequencing produces terabytes of data per experiment
- Engineering Simulations
 - Millions of elements modeled in fine detail
 - Requires solving large systems of equations repeatedly

When Do You Use an HPC?

- Doing this on a laptop:
 - Not enough disk space
 - Not enough processors
 - Not enough time

HPC at Northwestern - Quest Computing Cluster

- Provides computing for all researchers from a wide range of fields
- Enables the use of multiple powerful computers simultaneously
- 5,500 active users, 1231 compute nodes, 80,390 CPU cores, 304 GPU Cards, 750 research software, 78 knowledge base articles, 28 training recordings

Quest



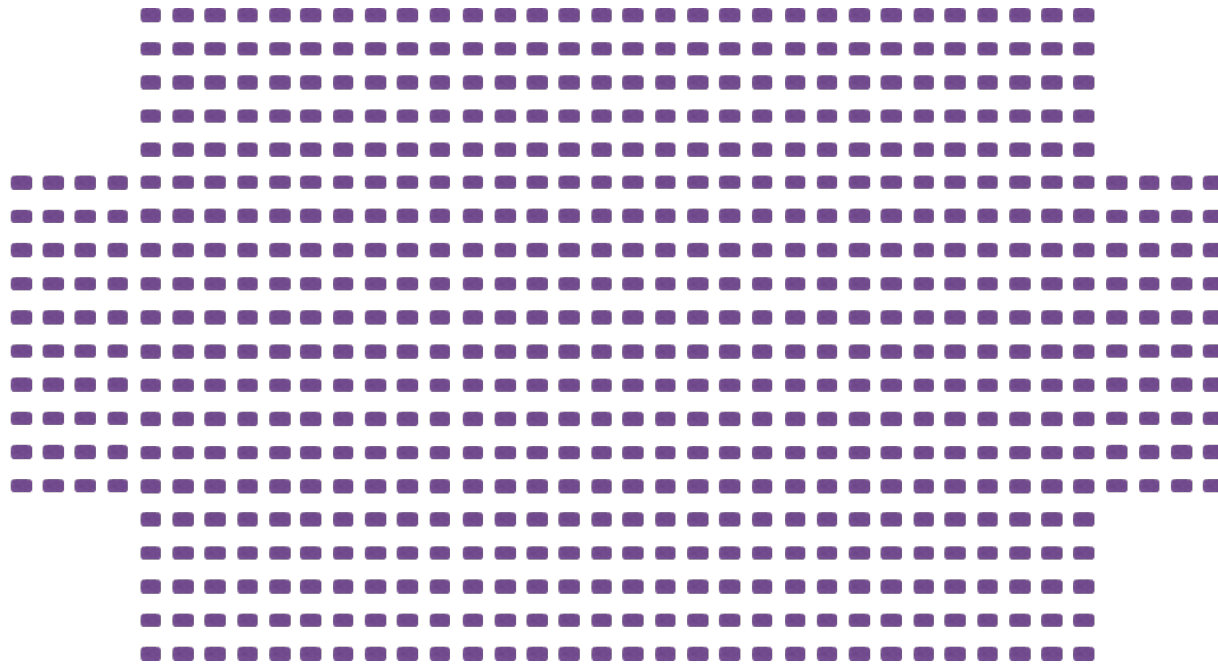
Other HPC clusters



Overview

1. What is High-Performance Computing?
2. Make-up of Quest
3. SLURM Job Scheduler
4. Types of Jobs
 - Submit a batch job to Quest
 - Submit a Quest OnDemand job
5. Checking on Jobs with Slurm
6. File transfers and Storage Practices
7. Software on Quest
8. TLDR

Quest consists of ~1200 nodes



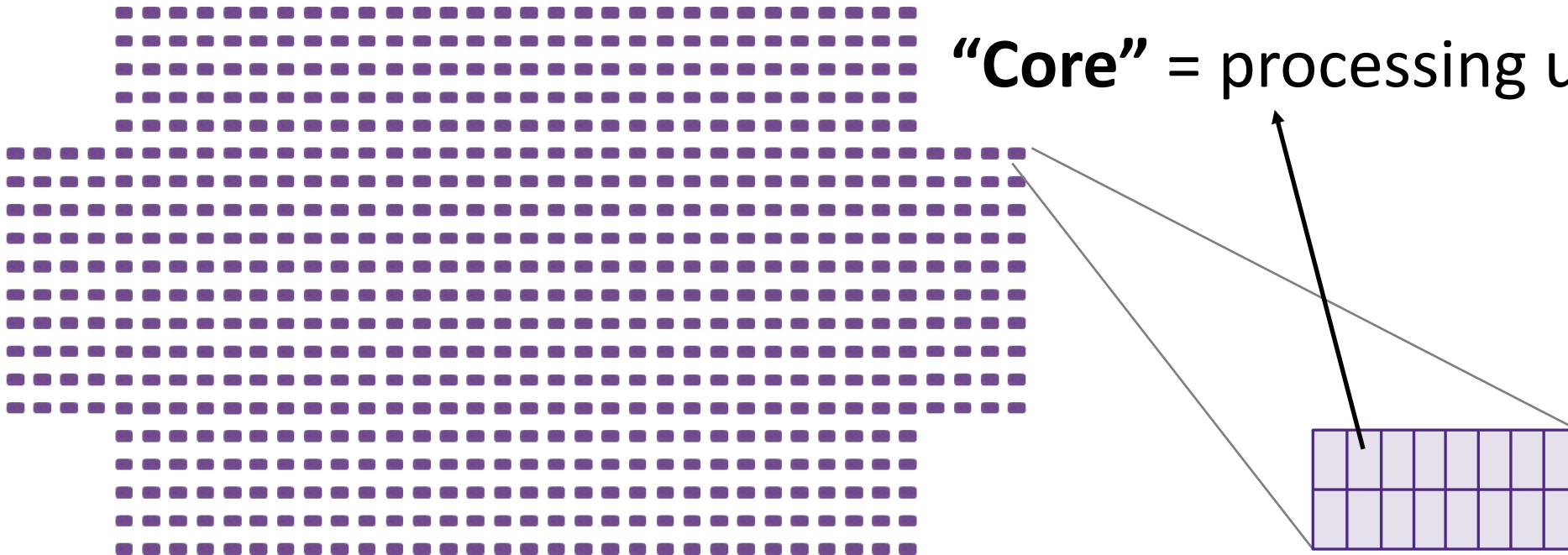
“Node” = computer



Each node consists of 52-128 cores

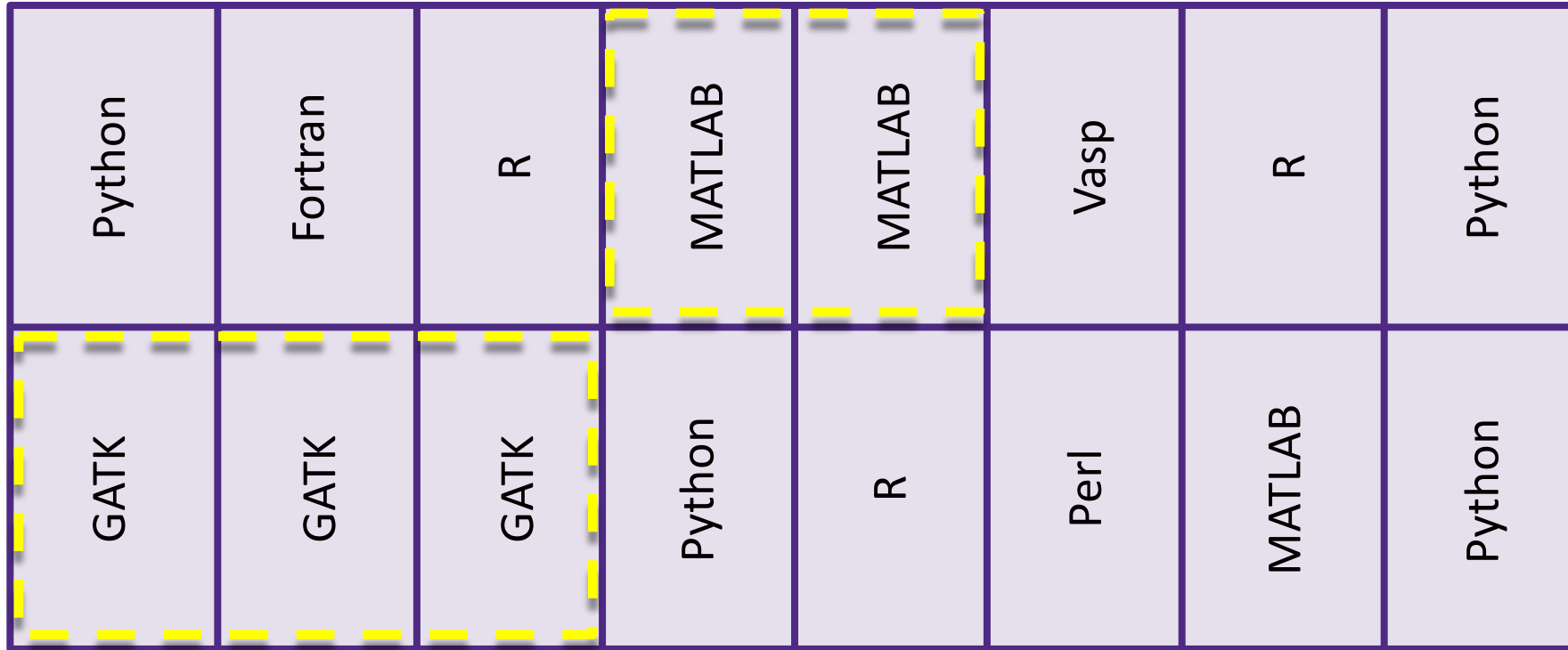
“Node” = computer

“Core” = processing unit within CPU



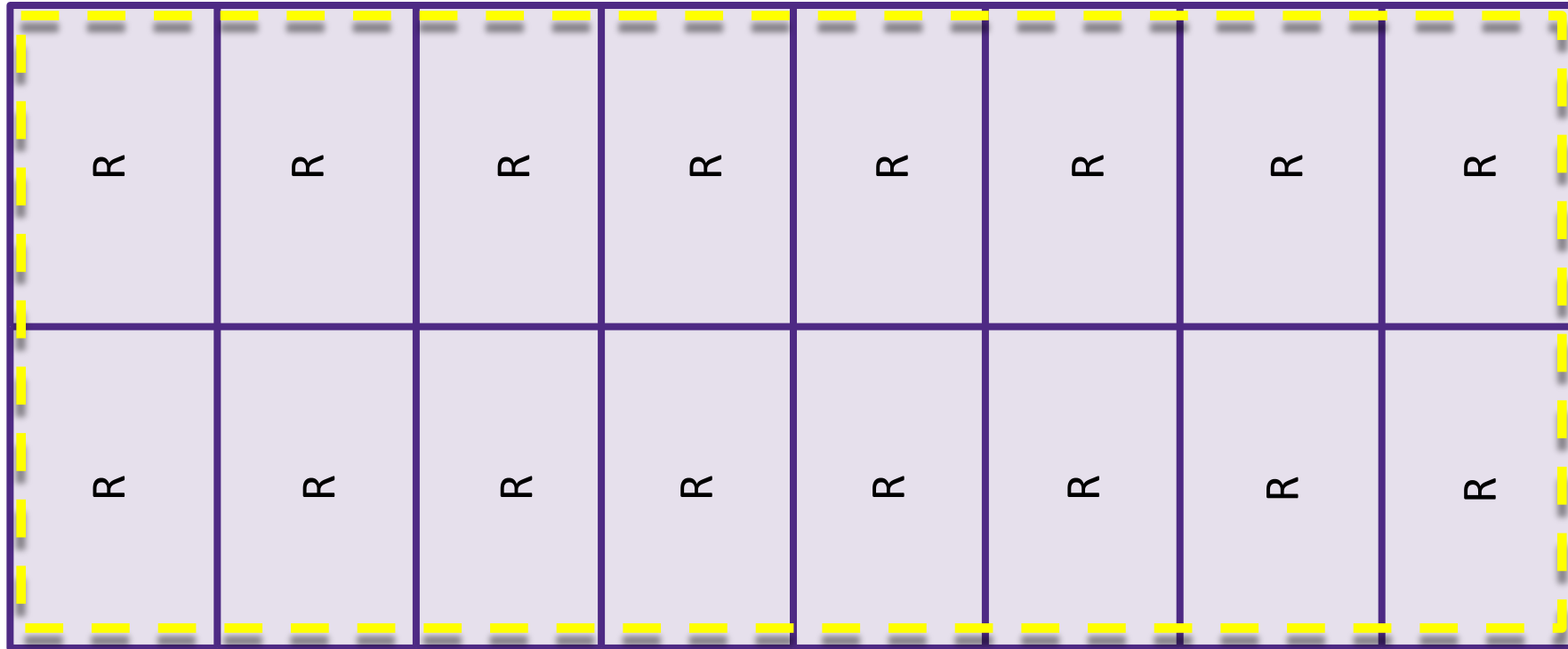
“Memory (RAM)” = Very fast temporary storage for processors

Each node can run separate jobs on each core



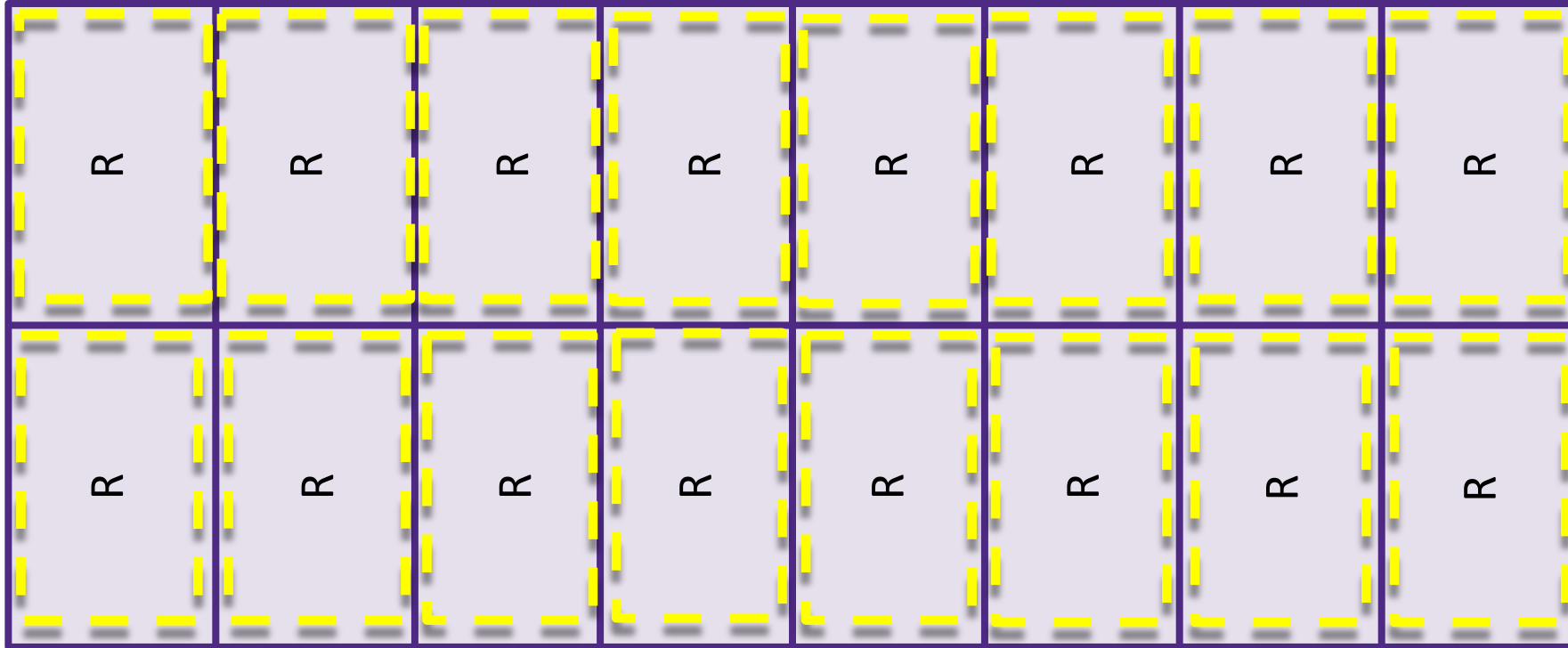
 = multi-core job

A node can run a job with 52-128 cores at a time



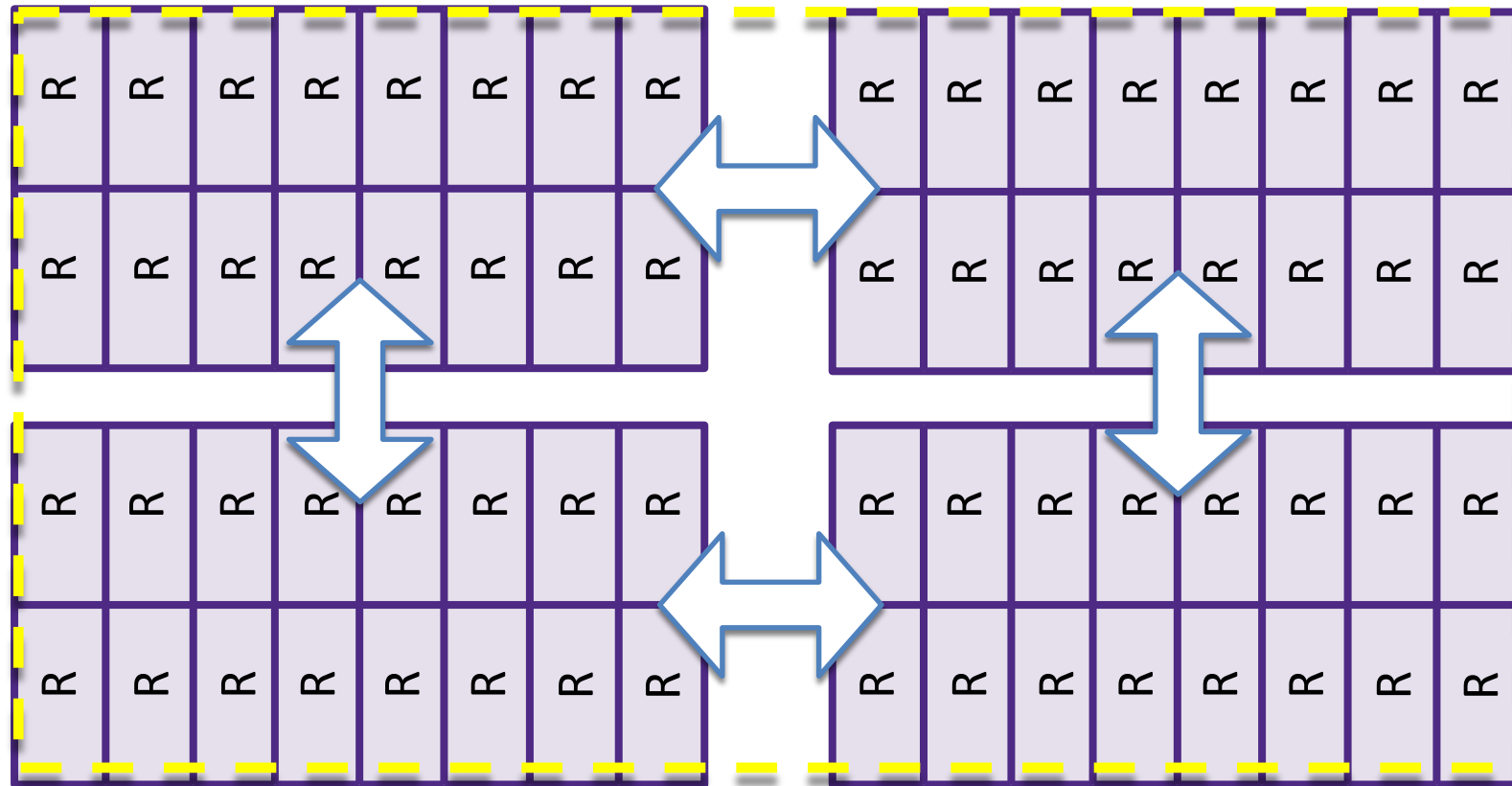
 = one multi-core job

You can run thousands of jobs at a time



 = a single job

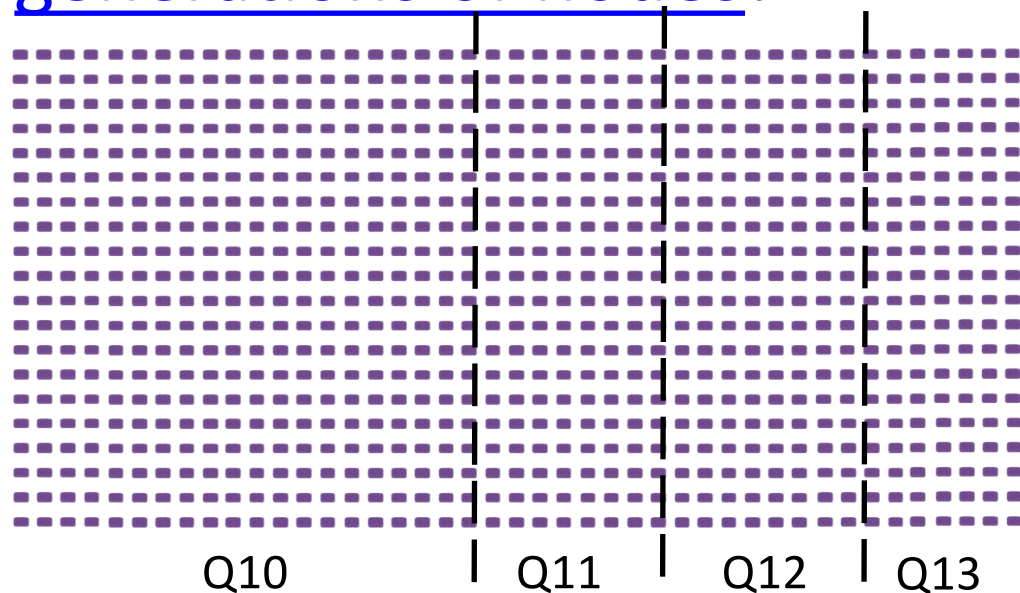
Jobs can even run across multiple nodes



 = a single job

Technical specifications of nodes vary

There are several
generations of nodes!

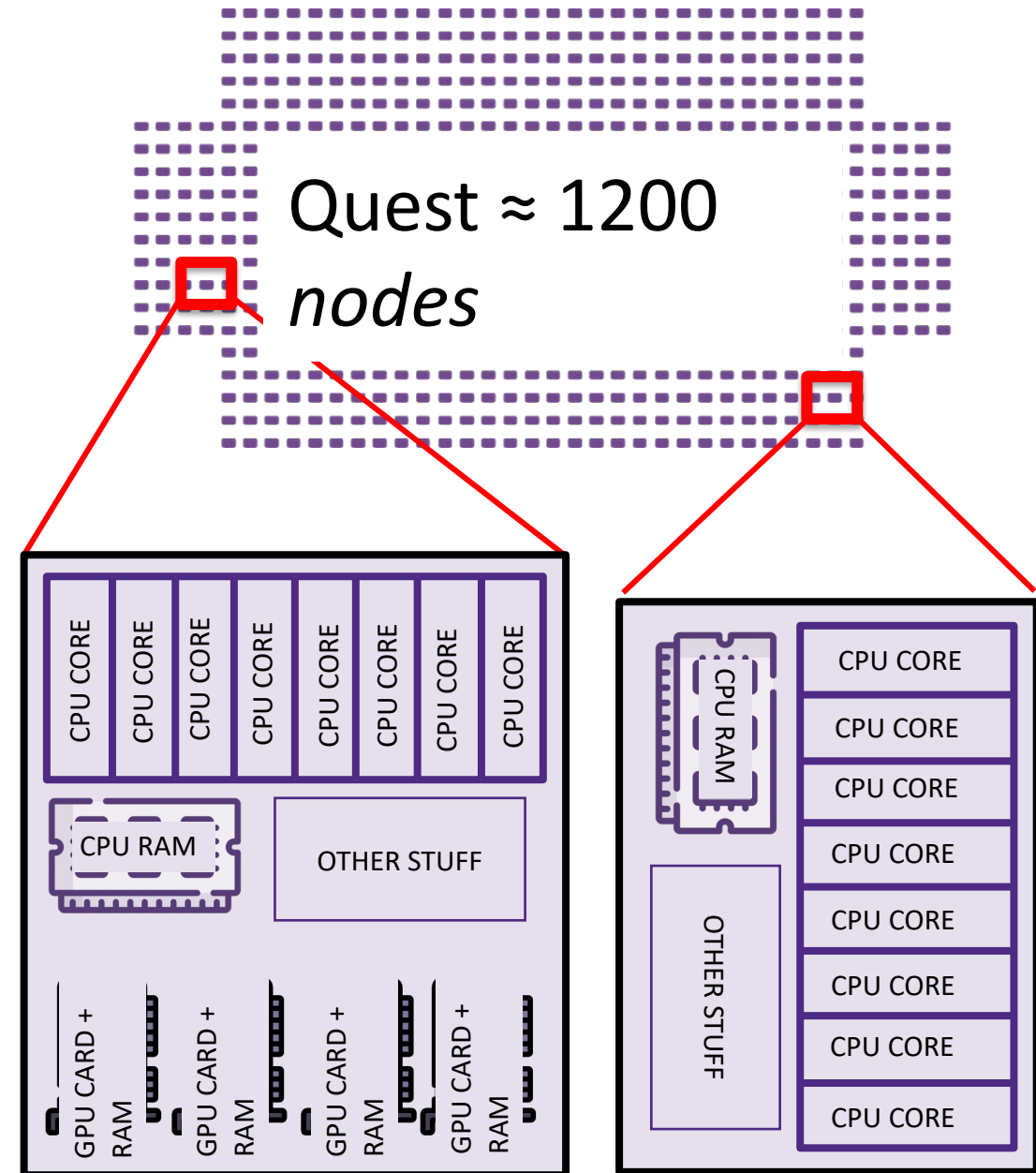


Generation	# nodes	# cores per node	Total usable memory (RAM)	Memory per core
Q10	555	52	166 GB	~3.2 GB
Q11	209	64	221 GB	~3.5 GB
Q12	214	64	221 GB	~3.5 GB
Q13	140	128	473 GB	~3.7GB
Your laptop (probably)	1	10-25	16-128GB	~2-8GB

Computing Resources

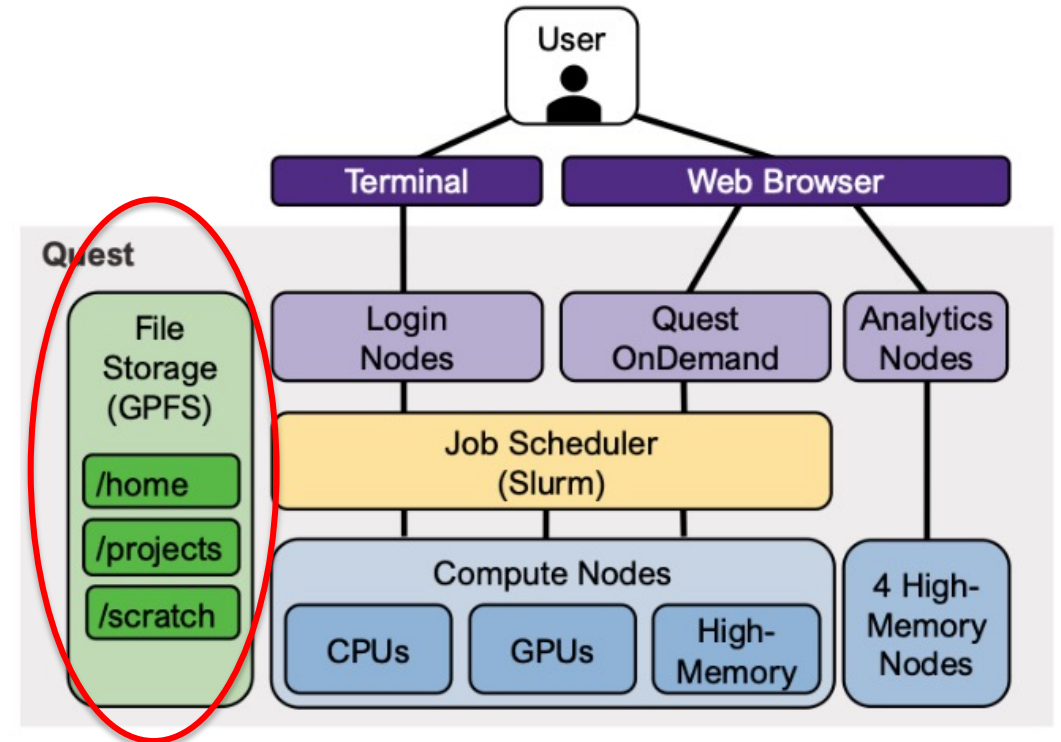
Nodes have schedulable resources:

- 52-128 *CPU cores*
- 192 - 1500 GB of *CPU RAM* (total)
- 0 - 4 *GPU cards*



Quest Filesystem and Working Directories

- /home - all users
 - 80 GB
 - Backed up
- /projects - all projects
 - 1 or 2 TB
 - Not backed up
- /scratch - upon request
 - 5 TB
 - Removed after 30 days



Overview

1. What is High-Performance Computing?
2. Make-up of Quest
3. SLURM Job Scheduler
4. Types of Jobs
 - Submit a batch job to Quest
 - Submit a Quest OnDemand job
5. Checking on Jobs with Slurm
6. File transfers and Storage Practices
7. Software on Quest
8. TLDR

Job Scheduler

- Job Scheduler = software that manages how computing resources are allocated to users and allocations on an HPC system
- Instead of users running programs directly on compute nodes, they submit jobs to the scheduler, which determines where, when, and how these jobs will run.
- Quest's job scheduler is SLURM
 - **S**imple **L**inux **U**tility for **R**esource **M**anagement



Job Scheduler

Order of operations:

1. User submits a job script
2. Scheduler places the job in a queue
3. Scheduler evaluates priorities and resource availability
4. Resources are allocated
5. Jobs runs on the assigned compute nodes
6. The results are returned and the resources are released

Overview

1. What is High-Performance Computing?
2. Make-up of Quest
3. SLURM Job Scheduler
4. Types of Jobs
 - Submit a batch job to Quest
 - Submit a Quest OnDemand job
5. Checking on Jobs with Slurm
6. File transfers and Storage Practices
7. Software on Quest
8. TLDR

Job Types

- Batch job
 - Jobs that are submitted via script
 - Good for: code that has a longer runtime/more resources
- Interactive job
 - Jobs that have a graphical user interface (GUI), or real-time shell access
 - Good for: debugging and prototyping

SBATCH Example

```
#!/bin/bash
```

```
#SBATCH --account=p12345
```

```
#SBATCH --partition=short
```

```
#SBATCH --nodes=1
```

```
#SBATCH --ntasks-per-node=1
```

```
#SBATCH --time=00:10:00
```

```
#SBATCH --mem=1G
```

```
#SBATCH --job-name=sample_job
```

```
#SBATCH --output=output.log
```

} sbatch
headers

```
module purge all
```

```
module load python-anaconda3
```

```
source activate /projects/intro/envs/slurm-py37-test
```

} Setup software
environment

```
python --version
```

```
python slurm_test.py
```

} Run your code

Job Submission and Monitoring

- Submit a job to the scheduler to run
 - `sbatch submission_script.sh`
- How do I check if my job is running?
 - `squeue -u <NETID>`

Job Resources

- Quest partitions:
 - `short` | `normal` | `long` | `gengpu`
- More memory | CPUs | GPUs | = longer queue wait time
- Make sure your code can use the resources:
 - Multi-CPU
 - Multi-GPU

Exercise: submit a batch job to run on Quest

- SSH to Quest
 - `ssh <NETID>@login.quest.northwestern.edu`
- **Navigate to your folder in /projects/e53003**
 - `cd /projects/e53003`
 - `cd firstname_lastname`
- **Clone the git repository https://github.com/nuitrcs/intro_quest_reu**
 - `git clone https://github.com/nuitrcs/intro_quest_reu.git`
- **Use a command line text editor to edit the submission script `submission_script.sh`**
 - `nano submission_script.sh`
- **Submit the job to run on Quest**
 - `sbatch submission_script.sh`
- **Examine the output**
 - `cat slurm-<JOBID>.out`

Exercise: submit a batch job to run on Quest

```
[abc1234@quser41 ~]$ cd /projects/e53003/<first_last>
```

```
[abc1234@quser41 ~]$ git clone  
https://github.com/nuitrcs/intro_quest_reu.git
```

```
[abc1234@quser41 ~]$ cd intro_quest_reu
```

```
[abc1234@quser41 ~]$ nano submission_script.sh
```

```
[abc1234@quser41 ~]$ sbatch submission_script.sh
```

```
[abc1234@quser41 ~]$ squeue -u <NETID>
```

```
[abc1234@quser41 ~]$ cat slurm-<JOBID>.out
```

Job Types

- Batch job
 - Jobs that are submitted via script
 - Good for: code that has a longer runtime/more resources
- Interactive job
 - Jobs that have a graphical user interface (GUI)
 - Good for: debugging and prototyping

Quest OnDemand

- Open OnDemand
 - Good for interactive development!
 - Requires VPN
 - Uses Quest partitions
- <https://ondemand.quest.northwestern.edu/>

Exercise: submit a Quest OnDemand job

- Navigate to <https://ondemand.quest.northwestern.edu/>
- Select the Jupyter Interactive Application
- Complete the job submission form using:
 - SLURM partition = `short`
 - SLURM account = `e53003`
 - Jupyter root directory = `/projects/e53003/`
- Run the notebook `happy_face.ipynb` in your clone of the `intro_quest_reu_github` repository with the kernel `ml-data-science-kernel-py311`

Exercise: submit a Quest OnDemand job

 RELION	↓ Pre-Installed Kernel
 SimBA	ml-data-science-kernel-py311
 Stata	↓ SLURM Partition
 uProbeX	short
Servers	↓ SLURM Account
 Jupyter	e53006
 Jupyter + Spark	↓ Number of CPUs/cores/processors
 Open WebUI	1
 PCEQuestApps	
 Platforma	

Kernels

- Creating your own Jupyter Kernels (to use in Jupyter or VSCode)
- <https://rcdsdocs.it.northwestern.edu/tutorials/software-management/conda-mamba-quest/mamba-conda-quest.html#accessing-mamba-conda-environments-in-jupyter>

Overview

1. What is High-Performance Computing?
2. Make-up of Quest
3. SLURM Job Scheduler
4. Types of Jobs
 - Submit a batch job to Quest
 - Submit a Quest OnDemand job
5. Checking on Jobs with Slurm
6. File transfers and Storage Practices
7. Software on Quest
8. TLDR

Checking running jobs: `queue`

- `queue -u <NETID>`
 - Shows the jobs in the queue for the given user.
 - Queued jobs have the status Pending or Running.
 - Pending jobs will list the reason they are waiting
 - Resources means you are next in line, but the resources aren't available
 - Priority means a different job in the queue has higher priority

Checking job history: sacct

- `sacct -X -S <startdate>`
 - Shows your job history.
 - Default to only today's history if no start date is provided.
 - Start dates should be provided in MMDDYY.
 - This is a good way to look up Job ID numbers.

Checking job resource use: seff / checkjob

- `seff <jobid>`
 - Shows a job's resource use efficiency
 - Only accurate after a job has completed
- `checkjob <jobid>`
 - Shows job details including, seff output
 - Also includes Slurm variables and the job script

Exercise: look at your Quest job history

- Try running the following commands:
 - `sacct -X`
 - `sacct -X --format=jobid,jobname%20,account,partition,submit,elapsed,state,nodelist`
 - `seff <ONE_OF_YOUR_RECENT_JOBIDS>`
 - `checkjob <ONE_OF_YOUR_RECENT_JOBIDS>`

Exercise: look at your Quest job history

```
[abc1234@quser41 ~]$ sacct -X
JobID          JobName Partition Account AllocCPUS      State ExitCode
-----
5499423      my_test_j+   short    e53006         1 COMPLETED      0:0
5499621      jupyter_j+   short    e53006         1 CANCELLED+      0:0

[abc1234@quser41 ~]$ sacct -X --format=jobid,jobname%20,account,partition,submit,elapsed,state,nodelist
## Output is long ##

[abc1234@quser41 ~]$ seff 5499423
Job ID: 5499423
Cluster: quest
User/Group: tdm5510/tdm5510
State: COMPLETED (exit code 0)
Cores: 1
CPU Utilized: 00:00:02
CPU Efficiency: 7.41% of 00:00:27 core-walltime
Job Wall-clock time: 00:00:27
Memory Utilized: 16.05 MB
Memory Efficiency: 0.39% of 4.00 GB (4.00 GB/node)

[abc1234@quser41 ~]$ checkjob 5499423
## Output is long ##
```

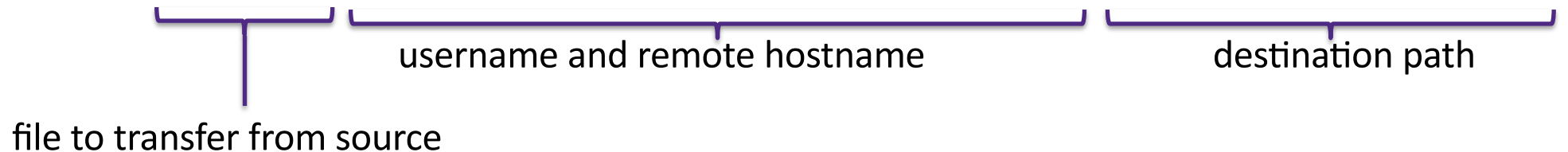
Overview

1. What is High-Performance Computing?
2. Make-up of Quest
3. SLURM Job Scheduler
4. Types of Jobs
 - Submit a batch job to Quest
 - Submit a Quest OnDemand job
5. Checking on Jobs with Slurm
6. File transfers and Storage Practices
7. Software on Quest
8. TLDR

Transferring Files - scp

- Command line options
 - scp
 - rsync
 - many others
- Allow users to copy files from one location to another in the command line

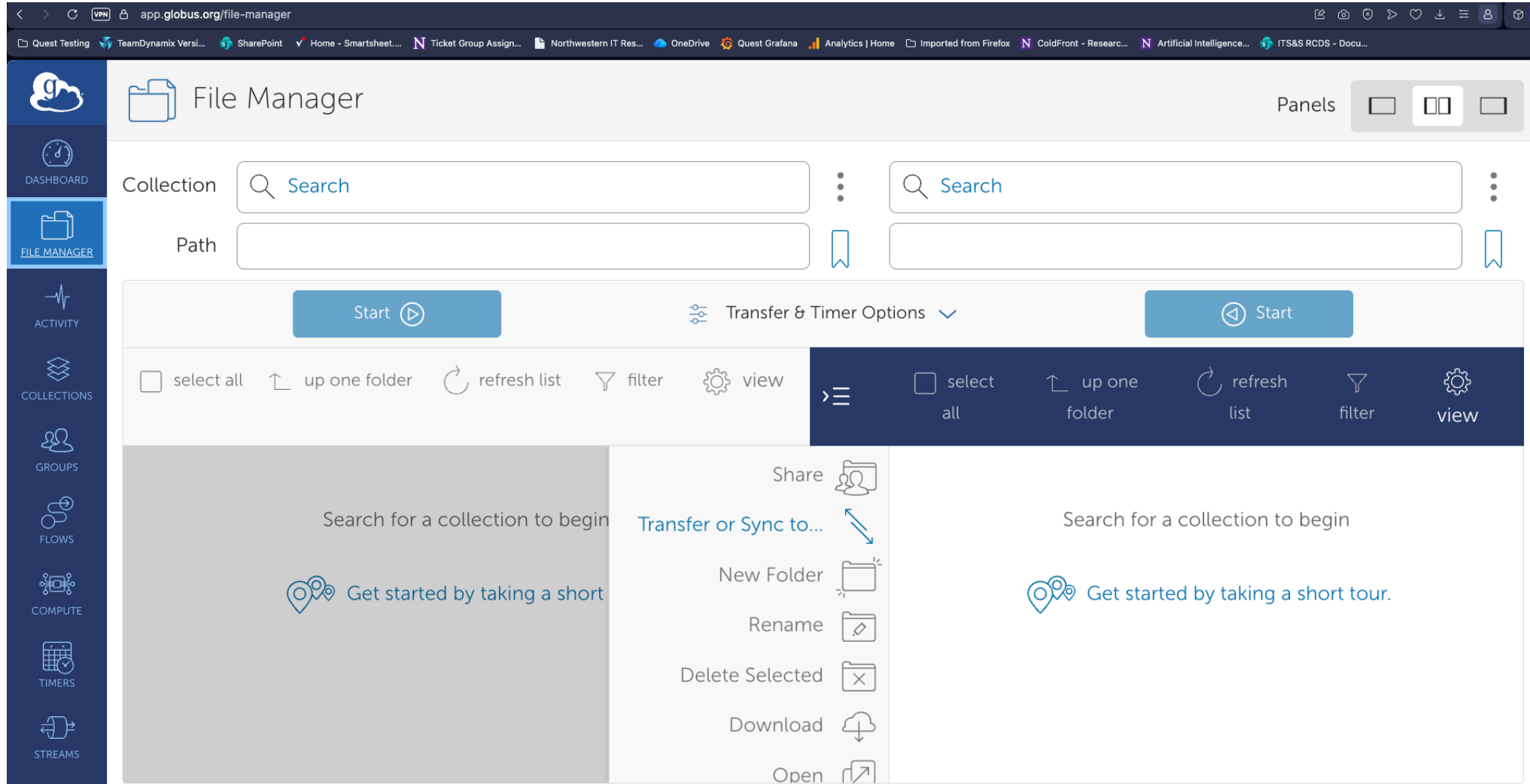
```
alice@laptop ~ % scp file.txt alice@quest.login.northwestern.edu:/path/to/destination/
```



Globus

- What's the best way to move data to Quest?
 - Globus!
- Quest, RDSS, and NU Sharepoint are Globus endpoints
- Many institutions and government agencies have Globus endpoints

Globus



Storage Best Practices

- Don't have more than 7,500 files in a single directory
- In aggregate, don't have more than 50 million files in a full directory path
- Helpful utilities
 - `homedu`
 - `checkproject`
 - `dust` (module)

Storage Best Practices

- Quest cannot store data subject to compliance frameworks (HIPAA, FERPA, NIST 800-171, NIST 800-53, FISMA, FedRAMP, etc.)
 - Contact quest-help@northwestern.edu with any questions

Overview

1. What is High-Performance Computing?
2. Make-up of Quest
3. SLURM Job Scheduler
4. Types of Jobs
 - Submit a batch job to Quest
 - Submit a Quest OnDemand job
5. Checking on Jobs with Slurm
6. File transfers and Storage Practices
7. Software on Quest
8. TLDR

Software on Quest

- Manage Python and R packages through virtual environments
- Can I compile my own applications?
 - Yes, you are responsible for understanding and obeying license requirements
- Can the RCS team help me compile an application?
 - Yes!
 - Submit a ticket to quest-help@northwestern.edu

Virtual Environments

- Access Python or R packages from within jobs submitted to the scheduler
- Improve portability and reproducibility
- Isolate dependencies per project
- Python
 - Use Conda or Mamba
- R
 - Use Conda or Mamba OR use renv

Overview

1. What is High-Performance Computing?
2. Make-up of Quest
3. SLURM Job Scheduler
4. Types of Jobs
 - Submit a batch job to Quest
 - Submit a Quest OnDemand job
5. Checking on Jobs with Slurm
6. File transfers and Storage Practices
7. Software on Quest
8. TLDR

Summary

- Quest (and HPC clusters in general) provides computation and storage resources for research workflows
 - More CPUs, memory, and storage than a personal laptop or workstation
- Be a conscientious user
 - Strive for efficient resource usage
 - Start small and prototype/test codes before scaling up
 - Design your software environments and code repositories with reproducibility in mind
- Reference our documentation and/or reach out to quest-help@northwestern.edu for help



Thank you!